

On-Device AI Copilot for Private Banking Wealth Advisors

Solution Overview: We propose a secure on-device AI assistant that indexes all relevant client/prospect documents (tax returns, estate plans, onboarding files, CRM data, etc.) and answers advisor queries in natural language. The system uses a local retrieval-augmented generation (RAG) pipeline: it ingests files, generates semantic embeddings, and stores them in a vector index. When the advisor asks a question, the assistant performs a semantic search over the local index, retrieves relevant document excerpts, and feeds them into an on-device LLM to produce summaries or reports. Crucially, **all processing and data storage remain on the user's machine**, ensuring compliance with financial privacy regulations.

System Architecture

The system follows a classic RAG workflow, entirely on-premises:

- **Document Ingestion:** A background service scans approved local storage (folder mounts, network drives, etc.) for client files (PDF, Word, CSV, CRM exports, etc.). Each document is parsed (e.g. using Apache Tika or PyMuPDF) to extract text and metadata. The content is converted into a structured format (Markdown is one option) to preserve formatting cues like headings and lists ¹. This preserved structure gives the embedding model richer context when encoding the text.
- **Text Chunking:** Extracted text is split into chunks that fit the LLM's context window ². A natural-language splitter (such as LangChain's chunker) breaks documents at sentence or paragraph boundaries, ensuring each chunk is self-contained. This prevents oversized prompts and helps the AI retrieve only the most relevant snippets ².
- **Embedding/Indexing:** Each text chunk is passed through a local embedding model to produce a fixed-dimensional vector. For example, Google's new *EmbeddingGemma* model (≈ 308 million parameters) is optimized for on-device use: it generates "high-quality embeddings" in under 200MB of RAM, even offline ³ ⁴. The resulting vectors (together with metadata like the chunk text, source file, and client ID) are stored in a vector database. We recommend a database like **Qdrant**, a Rust-based engine designed for real-time vector search ⁵. Qdrant can store payload metadata and supports hybrid filtering (e.g. by date or client) ⁶. Internally, the index uses a Hierarchical Navigable Small World (HNSW) graph to accelerate nearest-neighbor search ⁷ ⁸.
- **Semantic Search:** At query time, the user's natural-language prompt is also converted into an embedding using the same model. The system then performs a similarity search on the local index. (For best results, one can combine semantic search with traditional keyword search – studies show that a hybrid of BM25 + vector search yields higher recall on financial documents ⁹.) The top- k most similar document chunks are retrieved.

- **LLM Query Interface:** The retrieved text chunks provide grounding context. The assistant feeds the user’s original query plus these chunks into a local LLM. For instance, Google’s open **Gemma 3** models (1B or 4B parameters) are explicitly designed to “run fast, directly on devices” and even support very long contexts (128K tokens) ¹⁰. The LLM generates an answer or summary based on both the query and the client-specific data ⁸. It might, for example, summarize a client’s recent portfolio performance or compile a complete investment report. Because all documents and embeddings stay on-device, the model never exposes sensitive data.

The table below summarizes each pipeline component and example technologies:

Component	Function	Example Tools / Models
Document Ingestion	Scan local files; parse text and metadata; convert to structured text (Markdown).	Apache Tika, PyMuPDF, Pandas (for CSVs)
Text Chunking	Split long text into LLM-friendly chunks ² .	LangChain’s text splitter, NLTK/regex rules
Embedding Model	Generate vectors from text.	EmbeddingGemma ³ , Sentence-Transformers (MiniLM)
Vector Database	Store embedding vectors + metadata; enable fast similarity queries ⁵ .	Qdrant ⁵ , SQLite+pgvector, Faiss library ¹¹
Search / Retrieval	Find nearest embeddings for a query (semantic k-NN; optional hybrid with BM25) ⁹ .	Faiss/HNSW ⁷ , hybrid BM25+vector search
LLM Inference	Answer queries / summarize using retrieved context ⁸ .	Gemma 3 (quantized) ¹⁰ , Llama2 or Mistral on llama.cpp
UI Layer	Chat interface, side panels, or overlays for user interaction.	Electron/React desktop app, or native UI toolkit (WPF, SwiftUI, etc.)
Security	Encryption at rest, access controls, logging.	OS disk encryption, TPM/TEE secure enclave, audit logs
Integration	Data sync/connectors to CRM or DMS.	Secure API clients, offline sync tools (e.g. Dynamics offline profile ¹²)

Privacy-First AI Models

Every model and algorithm runs locally with privacy in mind:

- **On-Device Models:** We select compact open models. *Gemma 3* (Google) offers models as small as 1B parameters with full quantized builds ¹⁰. For embeddings, *EmbeddingGemma* (308M params) is explicitly built for offline use and “private, high-quality” embeddings ³. By running these on the advisor’s workstation (leveraging CPU/GPU or even Neural Engine), no data ever traverses the internet.
- **Model Efficiency:** Techniques like 8-bit/4-bit quantization and pruning are applied. For example, Gemma 3 provides ready-made quantized models ¹⁰. Even on modest hardware, inference is

fast and memory-light. Optional use of libraries like *llama.cpp* or *GPT4All* can accelerate on-device LLM inference. Since inference is local, latency is minimized, and the firm retains full control over model parameters (supporting internal audits).

- **Data Protection:** Sensitive data remains encrypted and ephemeral. The embedding store can reside in an encrypted database or on encrypted disk. Vector indices contain no raw PII other than the original text (which can be encrypted at rest). The system avoids any cloud/API calls; even queries are not logged centrally. If analytics are needed (e.g. which questions are asked), they are anonymized using techniques like differential privacy ¹³. As one design recommendation notes, “selective on-device inference” and privacy filters help minimize any risk of data leakage ¹³.
- **Compliance Measures:** We enforce least privilege (only index folders the advisor has permission to see). Role-based access ensures the assistant cannot access unrelated client data. All components adhere to the financial industry’s “explainable AI” and governance requirements ¹⁴. For example, audit trails record that an AI query was executed (but not its content) so supervisors can verify usage. These patterns align with the EU AI Act’s emphasis on transparency and accountability ¹⁴.

Integrating with CRM and Document Systems

To avoid breaking compliance while leveraging existing systems:

- **Secure Sync/Import:** We periodically pull data from the CRM into the local environment using secure methods. Many enterprise CRMs (e.g. Microsoft Dynamics 365) offer offline sync features so that field apps work from a local cache ¹². We would use such facilities (or official SDKs/APIs) to download only the needed client records and documents. For example, an approved export job can dump new client transactions into CSV/JSON locally. All transport is done over TLS and authenticated, and no AI queries themselves call back to the CRM cloud.
- **Vendor Connectors:** If live access is required (e.g. a document management API), a local middleware process can fetch data on demand into the index. It never sends PII to third parties. For instance, the system could use OAuth to connect to a cloud CRM, retrieve the advisor’s clients data, and store it encrypted on the device. This data is then indexed as above.
- **Compliance Safeguards:** We apply standard data-privacy controls. Only required fields are synced (data minimization). Personally identifiable information is pseudonymized or masked in the index if possible. All keys and API credentials are stored in the OS keychain. The design follows AI+CRM best practices: “collect only necessary information,” encrypt data in transit and at rest, and enforce strict access controls ¹⁵. Regular audits verify that no unauthorized data flows occur.

UI/UX Interaction Patterns

The assistant should feel like a natural tool in the advisor’s workflow:

- **Global Hotkey or Tray Icon:** The advisor can summon the assistant with a keyboard shortcut (e.g. Ctrl+Alt+Space) or by clicking a menu-bar/tray icon. This pops up a chat window overlay on top of any application. This is akin to system assistants and ensures fast access without context switching.

- **Sidebar / Panel:** Embedding the assistant as a side panel in core applications is effective. For example, a persistent left-hand pane (like ChatGPT Canvas) ¹⁶ or a collapsible right-hand pane (as Microsoft Copilot does in Office) ¹⁷ keeps the current workspace visible while allowing multi-turn interaction. In practice, the CRM or portfolio app could host an “AI Assistant” sidebar tab that displays the chat history and response.
- **Inline Contextual Help:** For editing documents or emails, inline overlays can suggest AI completions. (This follows patterns in Notion or Grammarly, where AI prompts appear as you type) ¹⁸. For example, when viewing a client’s report, the advisor might select a paragraph and press a key to see a concise summary tooltip. This minimizes disruption, providing “in-place” assistance tied to the specific text.
- **Multi-Modal Inputs (Optional):** While primarily text-driven, the assistant could accept voice commands or present charts/graphs summarizing data. Any such features would rely on local speech engines or plotting libraries, again ensuring no external calls. Notifications or desktop alerts (e.g. reminding an upcoming client meeting with a summary of their portfolio) are also possible, but would use local OS mechanisms to stay compliant.

By combining these elements—quick keyboard access, an always-available sidebar chat, and inline prompts—the design makes the AI feel omnipresent but non-intrusive. Early UX research highlights that persistent chat panels (left or right) enable “ongoing co-creation” with AI ¹⁶ ¹⁷, while inline prompts offer precision help in context ¹⁸. Our UI would likely mix these: a chat interface for broad queries and summaries, plus subtle inline hints for document editing.

Implementation Stack & Tradeoffs

Below is an illustrative stack with open-source options:

- **Programming Languages:**
 - *Backend/AI Pipeline:* Python is a natural choice, given libraries like Hugging Face Transformers, LangChain, sentence-transformers, and binding for vector DBs.
 - *Desktop UI:* Electron (TypeScript/JavaScript) allows cross-platform GUI development (Windows, Mac, Linux). Alternatively, native frameworks (C#/NET/WPF for Windows or SwiftUI for macOS) offer tighter OS integration and system-tray support.
- **Embedding Models:**
 - *Embeddings:* We can use *EmbeddingGemma* ³ for its efficiency. If not available, a HuggingFace SentenceTransformer (e.g. `all-MiniLM-L6-v2`, ~300M params) is a proven lightweight model. These can run in *onnxruntime* or PyTorch on-device.
 - *LLM:* We suggest Google’s *Gemma 3* (1B or 4B) ¹⁰, which is optimized for single-accelerator inference. Alternatively, Meta’s Llama2-7B or Mistral-7B models (quantized to 4/8-bit) run on modern GPUs/CPUs. These models can be served via frameworks like HuggingFace or llama.cpp (GGUF format).
- **Vector Search Engine:**
 - *Qdrant:* A full-featured option (supports filters, incremental updates, persistent storage) ⁵.
 - *SQLite + pgvector:* Embeds into a single file with familiar SQL querying. Good for smaller scale.

- **Faiss:** If maximum speed is needed, Facebook's Faiss library (GPU-accelerated) can index millions of vectors ¹¹, but lacks built-in metadata handling (you'd manage it separately).

A sample comparison:

Option	Advantages	Considerations
Qdrant	Designed for vector search, supports payload filtering and hybrid queries ⁶ . Scalable and actively maintained.	Requires running a service (e.g. Docker or binary).
SQLite+pgvector	Simple, no extra service. Queries can mix SQL and vector similarity.	Best for moderate data sizes; performance lags at multi-million vectors.
Faiss (HNSW)	Extremely fast ANN search ⁷ , supports vector quantization.	No persistence/metadata layer; more coding overhead.

- **Embedding Search:** For ANN, we rely on HNSW graphs ⁷. Both Qdrant and Faiss use HNSW by default for fast k-NN. We can tune recall/speed tradeoff via HNSW parameters (e.g. search depth).
- **RAG Orchestration:** Use a framework like LangChain or LlamaIndex to glue the pipeline. These let us define document loaders, chunkers, vector stores, and LLM chains declaratively.
- **Security & Compliance:** All chosen components are offline/Open Source (no hidden API calls). Data at rest is encrypted using OS/DB encryption (e.g. Windows BitLocker or encrypted SQLite). Use TLS for any optional network sync. Logging and audit libraries should be added to capture operational events for compliance.

By carefully selecting lightweight, local-capable models and combining them with a semantic index, we achieve a Copilot that can, for example, scan a client's year-to-date statement and produce a board-ready summary – all without ever sending data to the cloud. This design blueprint meets both the wealth advisor's productivity needs and the banking industry's strict privacy/compliance requirements ¹³ ¹⁴.

¹ ² ⁸ Part 3: Local AI Ingestion Pipeline: Building from Scratch | by John Wong | Medium

https://medium.com/@able_wong/local-ai-ingestion-pipeline-building-from-scratch-part-3-0d06adb52c35

³ ⁴ Introducing EmbeddingGemma: The Best-in-Class Open Model for On-Device Embeddings - Google Developers Blog

<https://developers.googleblog.com/en/introducing-embeddinggemma/>

⁵ ⁶ ¹¹ Top 7 Open-Source Vector Databases: Faiss vs. Chroma & More

<https://research.aimultiple.com/open-source-vector-databases/>

⁷ On-Device Vector Search | ObjectBox Docs

<https://docs.objectbox.io/on-device-vector-search>

⁹ ¹³ ijirt.org

https://ijirt.org/publishedpaper/IJIRT178847_PAPER.pdf

10 **Gemma 3: Google's new open model based on Gemini 2.0**

<https://blog.google/technology/developers/gemma-3/>

12 **Configure offline data synchronization | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/field-service/mobile/offline-data-sync>

14 **How Financial Services Can Harness LLMs Safely & Effectively | FinTech Magazine**

<https://fintechmagazine.com/news/how-financial-services-can-harness-llms-safely-effectively>

15 **AI and CRM Integration: Addressing Data Privacy and Security – Powerful dialer software for Sales teams in real estate home services and insurance . ProspectBoss**

<https://prospectboss.com/ai-and-crm-integration-addressing-data-privacy-and-security/>

16 17 18 **Where should AI sit in your UI?. Mapping emerging AI UI patterns and how... | by Sharang Sharma | UX Collective**

<https://uxdesign.cc/where-should-ai-sit-in-your-ui-1710a258390e?gi=190b9d2b00ed>